

APRENDIZAJE AUTOMÁTICO

Trabajo Práctico #2



Por

Franco Orellana
Thiago García
Máxima Pellerano

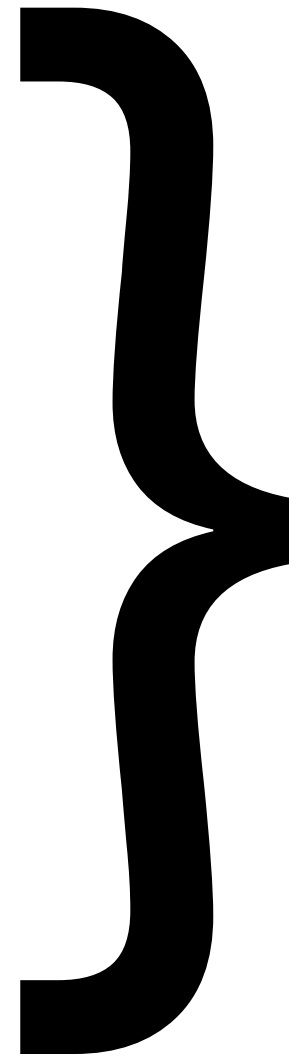


PROBLEMA

CRÉDITO BANCARIO ¿SI O NO?

Contamos con un conjunto de datos contiene información sobre personas que solicitaron un crédito bancario. Los datos que se conocen de cada persona son:

- Edad
- Sexo
- Mayor nivel educativo
- Ingreso anual
- Años de experiencia laboral
- Estado de vivienda
- Monto solicitado
- Destino de los fondos
- Tasa de interés
- Porcentaje del ingreso
- Plazo en años
- Puntaje
- Préstamos previos impagos
- **Estado**



De los cuales solo nos interesan los atributos **categoricos**, los cuales son:

- **Sexo**
- **Mayor nivel educativo**
- **Estado de Vivienda**
- **Préstamos previos impagos**
- **Estado** (Si se otorga el crédito o no)



SOLUCIÓN

Se desea conocer el estado de la solicitud del préstamo (**OTORGADO** o **RECHAZADO**) para personas de entre 40 y 45 años a partir de los atributos categóricos.

Para ello, vamos a hacer un programa que sea capaz de predecir si una persona entre esa edad va a recibir un préstamo o no.

Para realizar las predicciones, vamos a usar **2 algoritmos/métodos distintos**:

- **ID3** (con entropía de Shannon)
- **Random Forest**

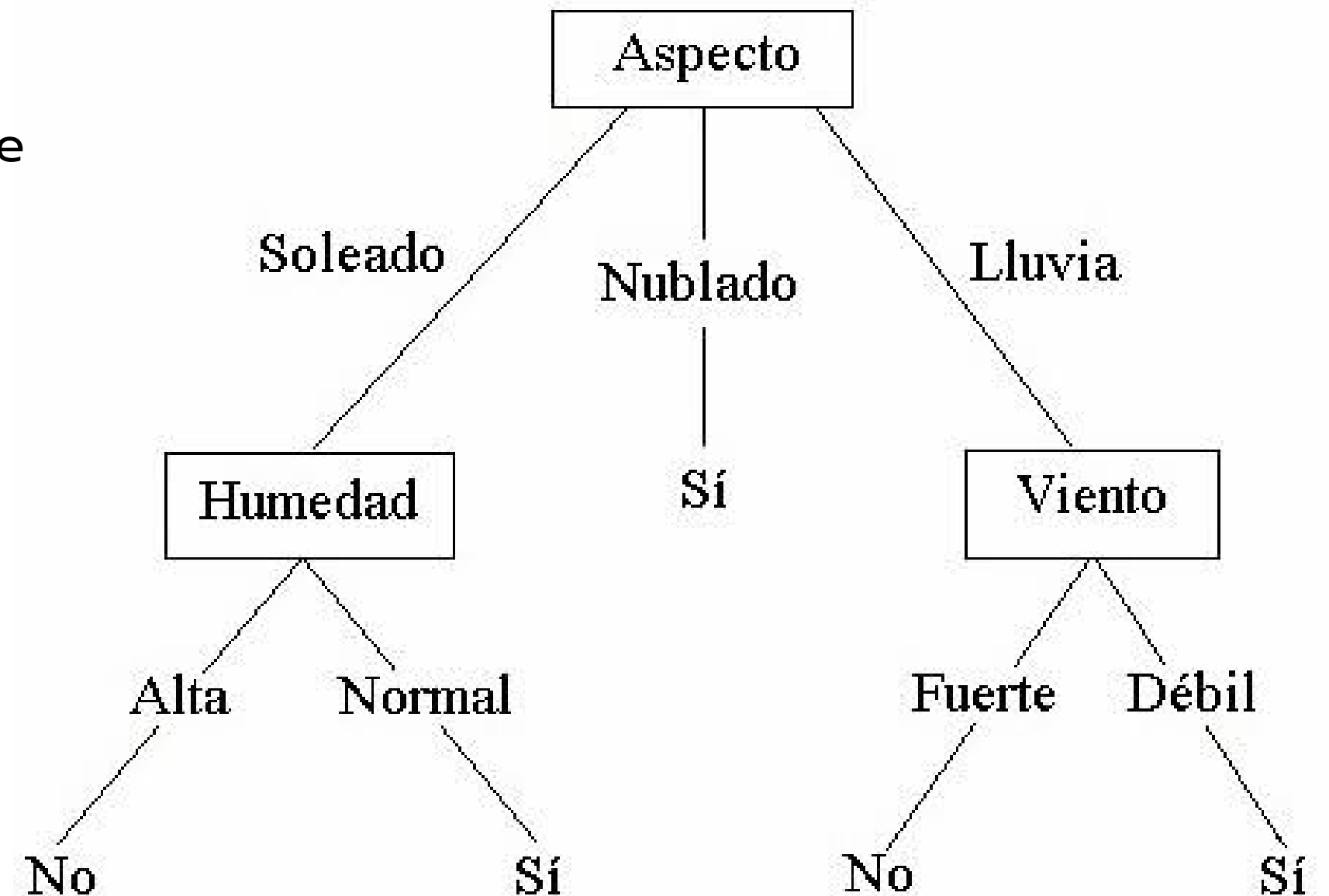


#1 PREDICCIÓN ID3

ID3

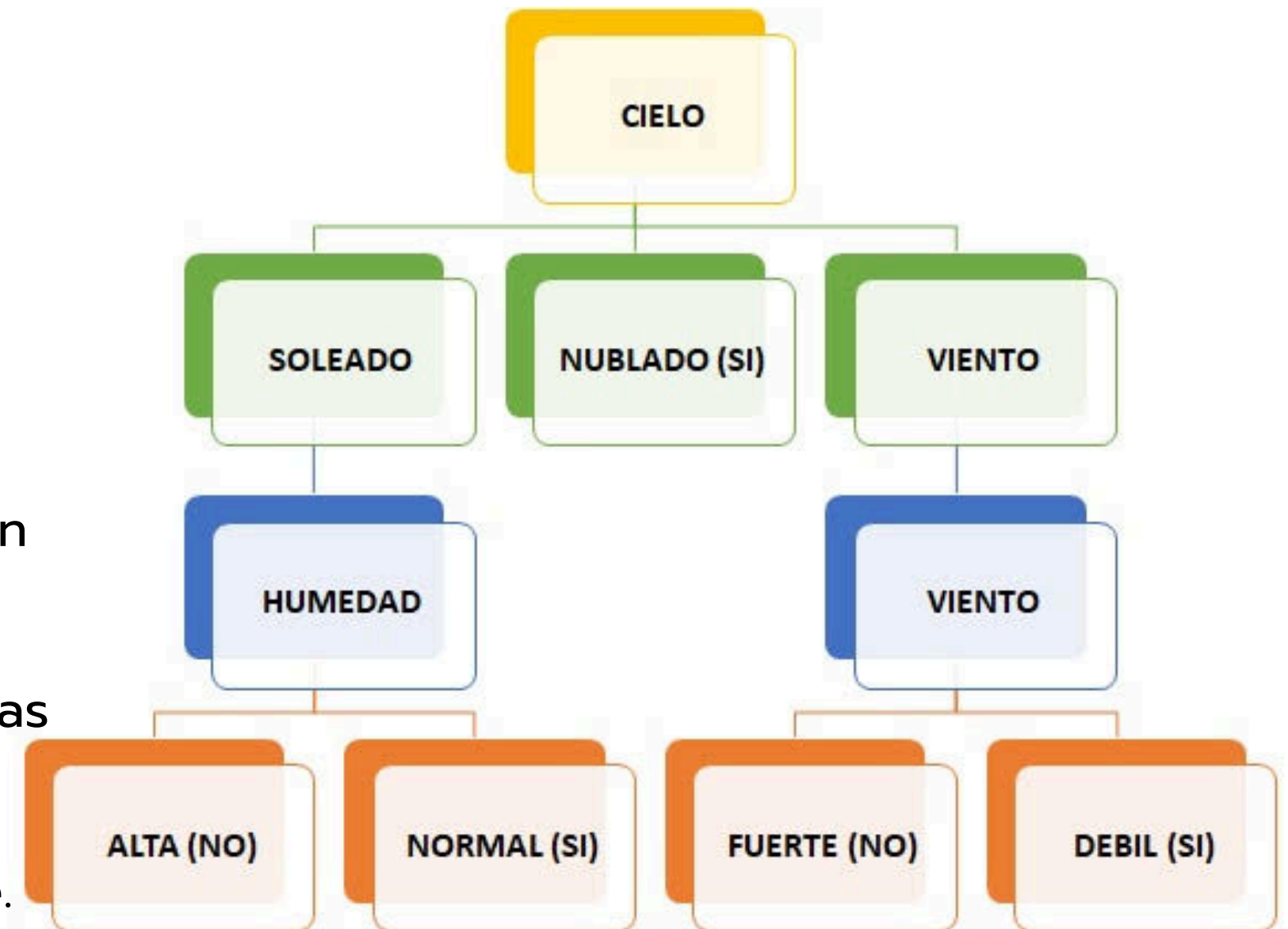
Método de aprendizaje automático supervisado que se utiliza para crear **árboles de decisión** a partir de un conjunto de datos.

- **Nodos de decisión:** Representan una pregunta sobre una característica o atributo de los datos.
- **Ramas :** Los posibles valores que puede tomar ese atributo.
- **Hojas:** Los nodos finales que indican la clase o categoría resultante de la clasificación



ID3 ¿CÓMO FUNCIONA?

- **Paso 1:** Calcular la **entropía** del conjunto de ejemplos de entrenamiento.
- **Paso 2:** Calcular la **ganancia de información** de cada atributo.
- **Paso 3:** Elegir el atributo con mayor ganancia de información.
- **Paso 4:** Particionar el conjunto de datos en tantos sub-nodos como posibles valores toma el atributo obtenido en el Paso 3 y agregar las ramas correspondientes.
- **Paso 5:** Repetir los Pasos 1 a 4 hasta que suceda una de las siguientes opciones:
 - Todos los ejemplos en el nodo sean de la misma clase.
 - No quedan atributos para utilizar.
 - El conjunto de datos es vacío.

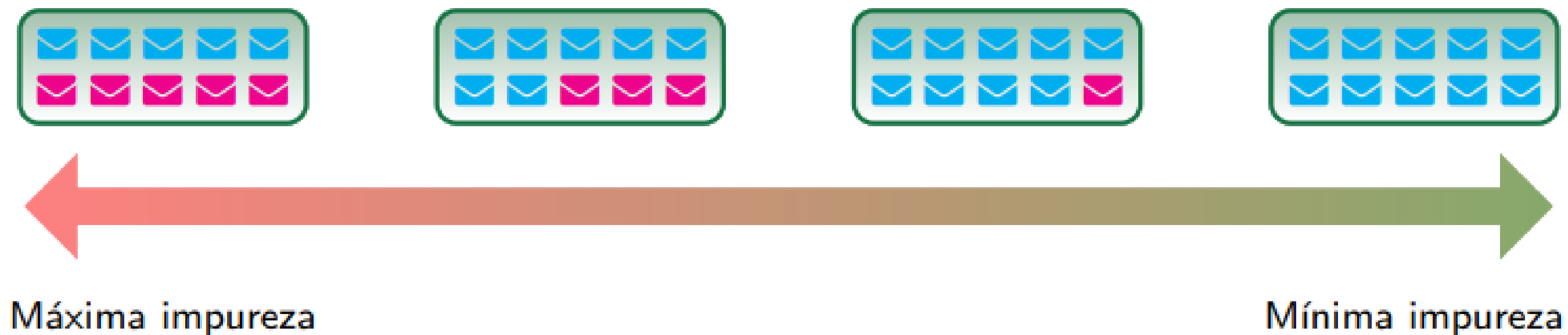


ID3 - ENTROPÍA DE SHANNON

Es una medida cuantitativa que evalúa el **grado de impureza**, incertidumbre o desorden contenido en un conjunto de datos **D**.

- **Entropía Mínima ($H = 0$):** El nodo es puro. Todos los ejemplos pertenecen a una misma clase.
- **Entropía Máxima ($H = 1$):** El nodo es impuro. Las distribución es aleatoria.

$$H(D) = - \sum_{k=1}^K p_k \log_2(p_k)$$



ID3 - ENTROPÍA CONDICIONAL DE SHANNON

Representa el **grado de impureza** que todavía permanece en los datos **D** después de dividirlos según los diferentes valores que puede tomar un atributo **A**

$$H(D|A) = \sum_{a \in \text{val}(A)} \frac{N(D_a)}{N(D)} H(D_a)$$

La entropía individual de cada subgrupo **D_a** correspondiente a un valor específico **a** de un atributo **A**
(Todos los valores posibles **a** que puede tomar un atributo **A**)

$$H(D_a) = - \sum_{k=1}^K \frac{N(k|a)}{N(D_a)} \log_2 \left(\frac{N(k|a)}{N(D_a)} \right)$$

```
def entropia_shannon(df, columna_objetivo):  
    """  
     $H(D) = - \sum(p_k * \log_2(p_k))$   
    Calcula la Entropía de Shannon  
    Si un  $p_k$  es 0, se considera nulo ( $\lim_{x \rightarrow 0} x * \ln(x) = 0$ )  
    """  
  
    prioris = calcular_prioris(df, columna_objetivo)  
    h = 0.0  
    for p in prioris.values():  
        if p > 0:  
            h -= p * math.log2(p)  
    return h
```

```
def entropia_condicional(df, atributo, columna_objetivo):  
    """  
     $H(D|A) = \sum( (N(D_a) / N(D)) * H(D_a) )$   
    Calcula la entropía ponderada de los nodos hijos  $H(D|A)$ :  
    """  
  
    total_ejemplos = len(df)  
    if total_ejemplos == 0:  
        return 0.0  
  
    h_cond = 0.0  
    valores_atributo = df[atributo].unique()  
  
    for val in valores_atributo:  
        sub_df = df[df[atributo] == val]  
        peso = len(sub_df) / total_ejemplos  
        h_cond += peso * entropia_shannon(sub_df, columna_objetivo)  
  
    return h_cond
```

ID3 - GANANCIA DE INFORMACIÓN

Mide la **reducción de entropía/incertidumbre** que se logra al particionar el conjunto de datos **D** según los valores de un atributo específico **A**

En cada etapa, el algoritmo ID3 evalúa todos los atributos disponibles y selecciona como nodo de decisión a aquel que maximiza la Ganancia de Información (**argmax IG(A)**)

$$IG(A) = H(D) - H(D|A)$$

$$IG(A1) = \dots$$

$$IG(A2) = \dots$$

$$IG(A_n) = \dots$$

Nos quedamos con el valor máximo, que va a representar que sus nodos hijos van a ser mas puros.

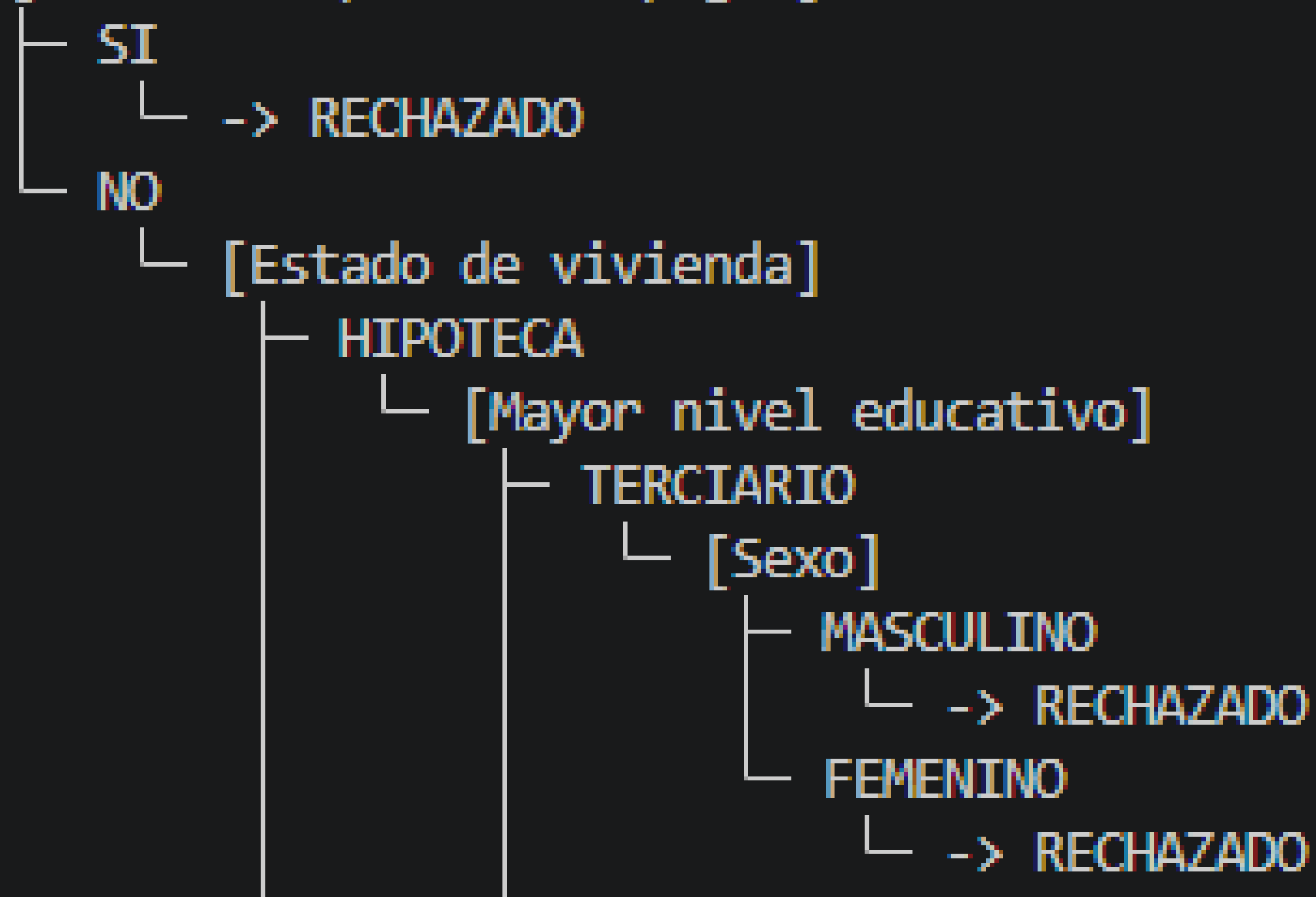
```
def ganancia_informacion(df, atributo, columna_objetivo):  
    """  
     $IG(A) = H(D) - H(D|A)$   
    Calcula la Ganancia de Información (Information Gain)  $IG(A)$   
    """  
    h_padre = entropia_shannon(df, columna_objetivo)  
    h_hijos = entropia_condicional(df, atributo, columna_objetivo)  
    return h_padre - h_hijos
```

ID3 - PARTICIONAR Y REPETIR

Usamos el atributo elegido en el paso anterior para **particionar** nuestros datos segun los valores posibles **a** que puede tomar dicho atributo **A**.

Vamos a **repetir los Pasos 1 al 4** en cada una de las ramas hasta llegar a alguna de las condiciones de corte mostradas anteriormente.

Estructura del Árbol de Decisión ID3 Generado:
[Préstamos previos impagos]



ID3 - EVALUACIÓN, RESULTADOS Y MÉTRICAS

Matriz de confusión:

- Ejemplos totales: 1426
- Set de Entrenamiento (80%): 1141
- Set de Prueba (20%): 285

Matriz de confusión:

- **Verdaderos Positivos (TP):** 22
- **Verdaderos Negativos (TN):** 206
- **Falsos Positivos (FP):** 20
- **Falsos Negativos (FN):** 37

Métricas de evaluación:

- **Accuracy:** 80%
- **F1-Score:** 43.56%
- **Recall:** 37.29%
- **Precision:** 52.38%

ALGORITMO ID3 (ARBOLES DE DECISIÓN)

Ejemplos totales (40-45 años): 1426

Set de Entrenamiento (80%): 1141

Set de Prueba (20%): 285

MATRIZ DE CONFUSIÓN (Conjunto de prueba)

Verdaderos Positivos (TP): 22

Verdaderos Negativos (TN): 206

Falsos Positivos (FP): 20

Falsos Negativos (FN): 37

MÉTRICAS DE EVALUACIÓN (Conjunto de prueba)

Accuracy: 80.00%

F1-Score: 43.56%

Recall: 37.29%

Precision: 52.38%

ID3 - CONCLUSIONES

- **Sesgo hacia la clase mayoritaria:** El algoritmo ID3, al maximizar la Ganancia de Información sobre un dataset desbalanceado (80/20), tiende a crear ramas y reglas que priorizan clasificar correctamente a la clase dominante (**RECHAZADO**)
- **Alta tasa de Falsos Negativos:** Para el negocio bancario, esto significa rechazar sistemáticamente a clientes que sí habrían pagado su deuda.

MATRIZ DE CONFUSIÓN (Conjunto de prueba)

Verdaderos Positivos (TP): 22
Verdaderos Negativos (TN): 206
Falsos Positivos (FP): 20
Falsos Negativos (FN): 37

MÉTRICAS DE EVALUACIÓN (Conjunto de prueba)

Accuracy: 80.00%
F1-Score: 43.56%
Recall: 37.29%
Precision: 52.38%

ID3 - PRE-PODA

El árbol sin podar estaba sobreajustando a la clase mayoritaria, “**memorizando**” los **RECHAZADOS** en lugar de reglas generales. Al limitar profundidad y exigir más ejemplos por nodo, el árbol quedó con reglas más simples y generalizó mejor

```
# Hiperparámetros de PREPODA
PROFUNDIDAD_MAX = 3
DIVISION_MINIMA = 10
GANANCIA_MINIMA = 0.005
```

MATRIZ DE CONFUSIÓN (Conjunto de prueba)

```
Verdaderos Positivos (TP): 37
Verdaderos Negativos (TN): 197
Falsos Positivos (FP):    29
Falsos Negativos (FN):    22
```

MÉTRICAS DE EVALUACIÓN (Conjunto de prueba)

```
Accuracy: 82.11%
F1-Score: 59.20%
Recall: 62.71%
Precision: 56.06%
```

ID3 - PRE-PODA

Seguimos probando distintos hiperparámetros para alcanzar distintos resultados.

```
# Hiperparámetros de PREPODA  
PROFUNDIDAD_MAX = 2  
DIVISION_MINIMA = 20  
GANANCIA_MINIMA = 0.01
```

MATRIZ DE CONFUSIÓN (Conjunto de prueba)

```
Verdaderos Positivos (TP): 42  
Verdaderos Negativos (TN): 186  
Falsos Positivos (FP): 40  
Falsos Negativos (FN): 17
```

MÉTRICAS DE EVALUACIÓN (Conjunto de prueba)

```
Accuracy: 80.00%  
F1-Score: 59.57%  
Recall: 71.19%  
Precision: 51.22%
```

#2

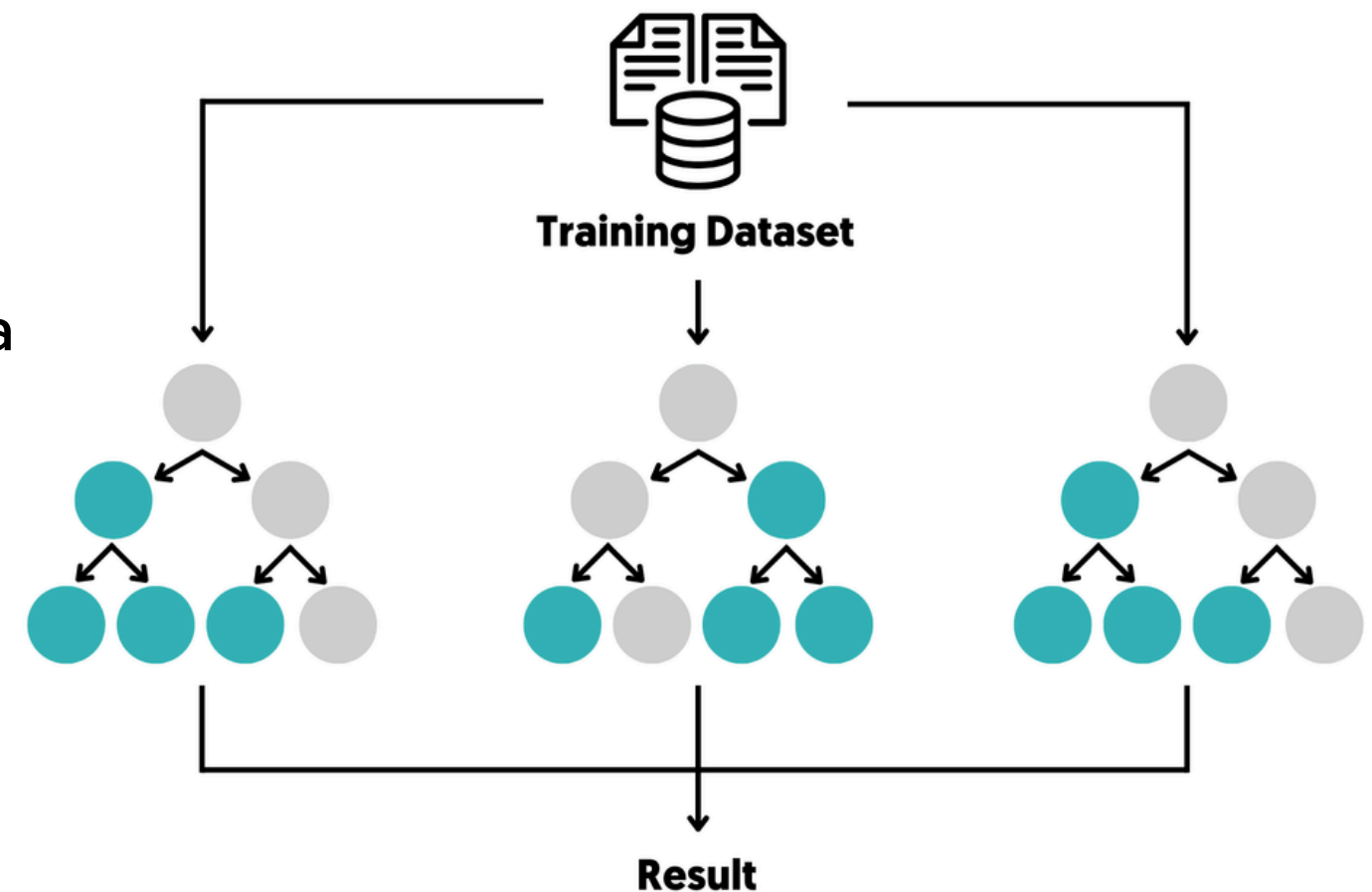
PREDICCIÓN

RANDOM FOREST

RANDOM FOREST

Método de aprendizaje supervisado que combina las predicciones de múltiples árboles de decisión individuales para tomar una decisión final, aplicando la filosofía del agregado (**aggregating**)

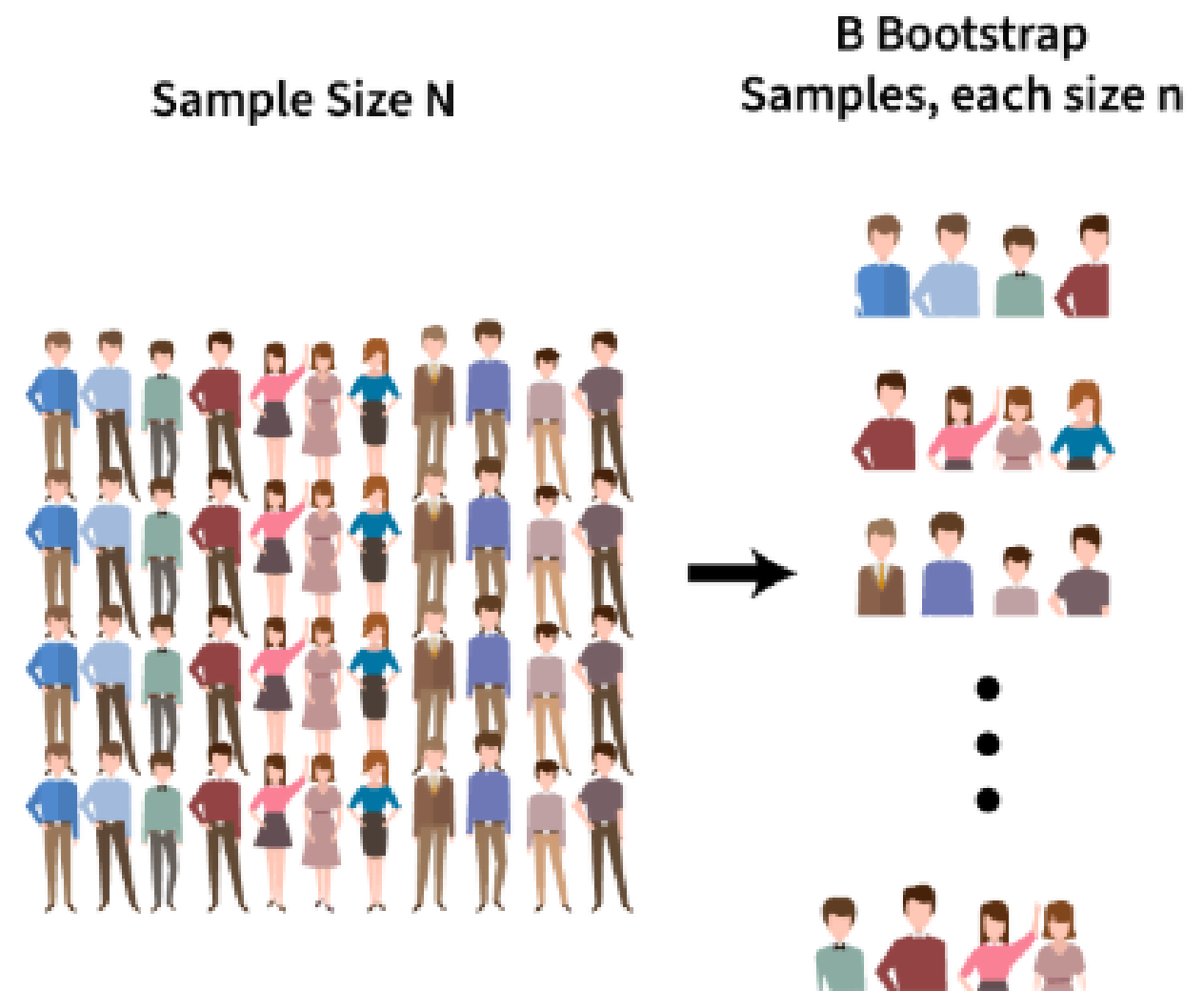
- Cada árbol emite su propia predicción.
- La predicción final del bosque se determina por voto de la mayoría.
- Usa un algoritmo de **Bagging (Bootstrap + Aggregating)** para entrenar a los distintos arboles del bosque.



BOOTSRAP

Técnica de remuestreo que consiste en extraer múltiples subconjuntos de datos a partir de una muestra original con reemplazo

- Se toma el conjunto de datos original de tamaño N .
- Se seleccionan observaciones al azar de forma repetida hasta completar otro conjunto de tamaño N .
- Al ser **con reemplazo**, un mismo dato u observación puede aparecer varias veces en el nuevo subconjunto y otros pueden quedar fuera.



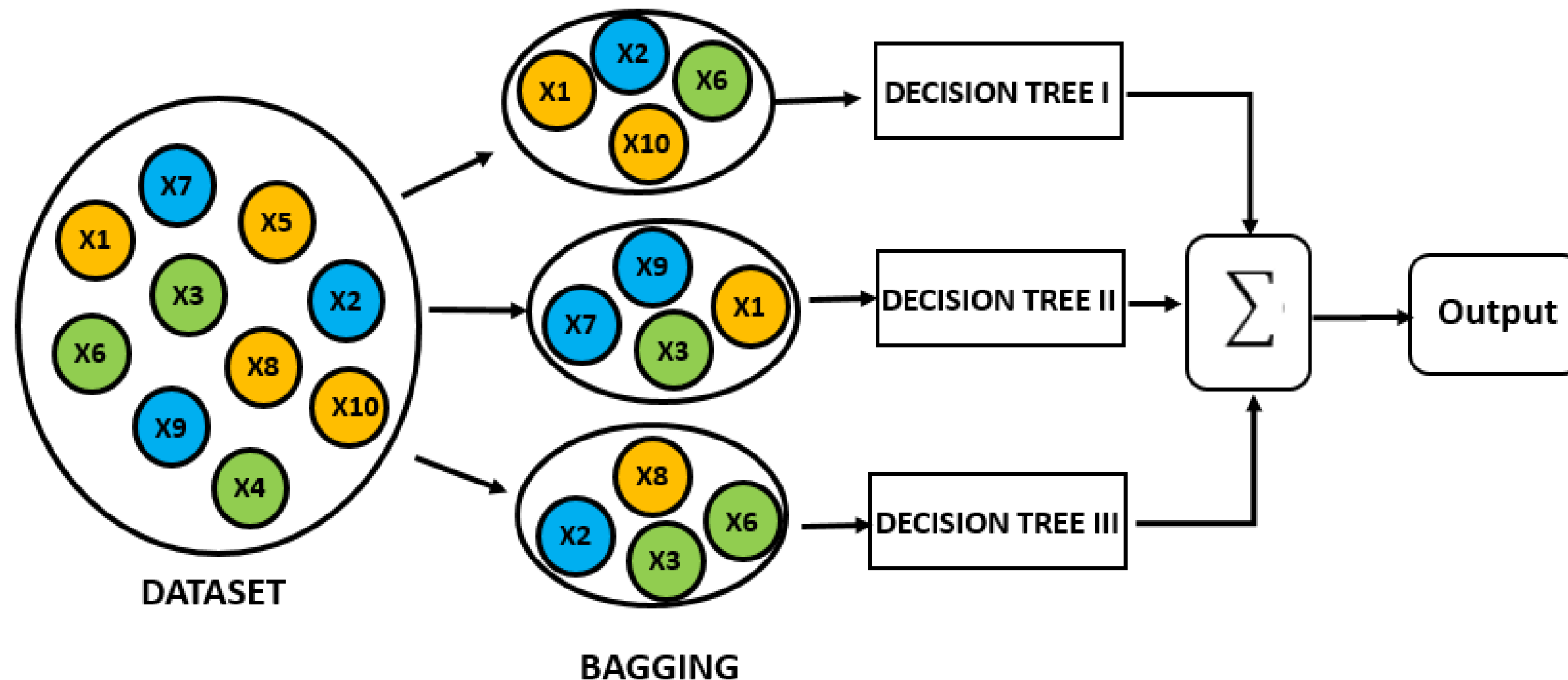
ALGORITMO DE BAGGING

1. Elegir una muestra aleatoria de tamaño N con reemplazo (bootstrap).
2. Construir un árbol de decisión sin podar.
3. Determinar la clase de cada hoja y registrar la clase a cada ejemplo nuevo.
4. Repetir los pasos 1 , 2 y 3 una gran cantidad de veces.
5. Para cada ejemplo nuevo, calcular la proporción de árboles que le asignan cada una de las clases de interés.
6. Asignar a cada nueva observación la clase mayoritaria obtenida.

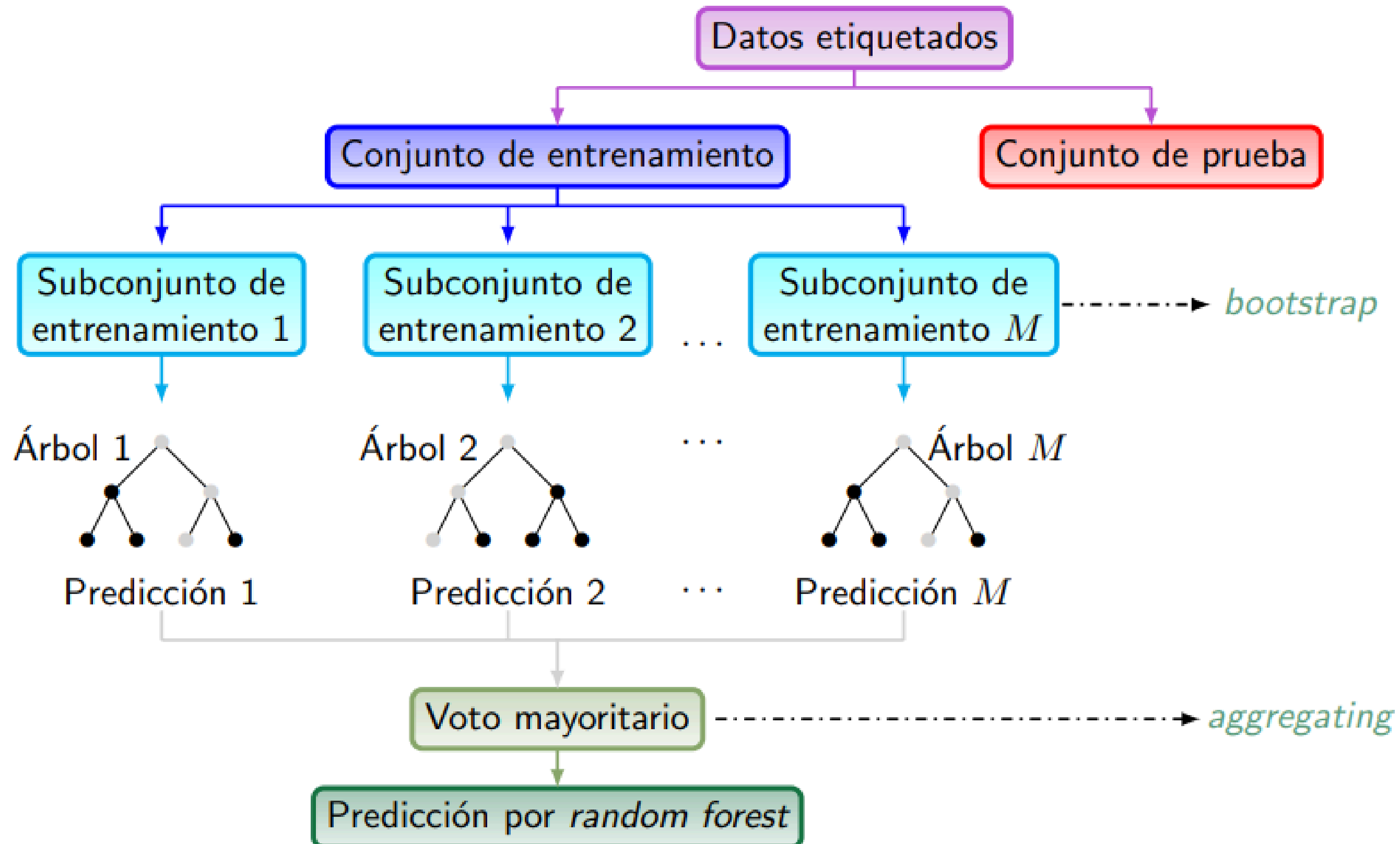
```
def generar_muestra_bootstrap(df, random_state=None):  
    return df.sample(n=len(df), replace=True, random_state=random_state)  
  
def muestrear_atributos_aleatorios(atributos_disponibles, num_atributos_muestreados):  
    cantidad_a_tomar = min(num_atributos_muestreados, len(atributos_disponibles))  
    return random.sample(atributos_disponibles, cantidad_a_tomar)
```

ALGORITMO DE RANDOM FOREST

1. Elegir una muestra aleatoria de tamaño N con reemplazo (bootstrap).
2. Elegir una muestra aleatoria, sin reemplazo, de los atributos.
3. Construir un árbol de decisión sin podar con los ejemplos del paso 1 y los atributos del paso 2 .
4. Determinar la clase de cada hoja y registrar la clase a cada ejemplo nuevo.
5. Repetir los pasos 1 , 2, 3 y 4 una gran cantidad de veces.
6. Con cada ejemplo nuevo, calcular cuantos árboles le asignan cada una de las clases de interés.
7. Asignar a cada nueva observación la clase mayoritaria obtenida.



ALGORITMO DE RANDOM FOREST



```
bosque = []  
for i in range(numero_arboles):  
    # BAGGING: una muestra bootstrap distinta para cada árbol  
    muestra_bootstrap = generar_muestra_bootstrap(df, random_state=semilla + i)  
    # ENTRENAMIENTO: cada arbol se entrena con su propia muestra  
    arbol = construir_arbol_id3_aleatorio(  
        muestra_bootstrap, atributos, columna_objetivo,  
        num_atributos_muestreados=num_atributos_muestreados,  
        tamano_nodo=tamano_nodo,  
    )  
    bosque.append(arbol)  
  
return bosque
```

ALGORIMO RANDOM FOREST

Ejemplos totales (40-45 años): 1426

Set de Entrenamiento (80%): 1141

Set de Prueba (20%): 285

RANDOM FOREST

Número de árboles: 10

Atributos muestreados por división (mtry): 2 de 4 totales

Tamaño mínimo de nodo: 1

MATRIZ DE CONFUSIÓN (Set de prueba, Random Forest)

Verdaderos Positivos (TP): 23

Verdaderos Negativos (TN): 202

Falsos Positivos (FP): 24

Falsos Negativos (FN): 36

Accuracy: 78.95%

F1-Score: 43.40%

Recall: 38.98%

Precision: 48.94%

Votación del bosque para el ejemplo #0

Árbol 1: OTORGADO
Árbol 2: RECHAZADO
Árbol 3: OTORGADO
Árbol 4: OTORGADO
Árbol 5: OTORGADO
Árbol 6: OTORGADO
Árbol 7: OTORGADO
Árbol 8: OTORGADO
Árbol 9: OTORGADO
Árbol 10: OTORGADO

Resumen de la votación:

OTORGADO: 9/10 votos (90.0%)

RECHAZADO: 1/10 votos (10.0%)

Predicción final del bosque: OTORGADO

RANDOM FOREST - CONCLUSIONES

- El bosque se vuelve extremadamente sesgado hacia el "**RECHAZADO**"
- Bajo Recall, por lo que no esta capturando los casos positivos.
- Un F1-Score bajo que indica que no hay equilibrio entre ambas clases

Posibles causas:

- Baja cantidad de arboles (10)
- Baja cantidad de atributos (2 por cada arbol)

```
=====
MATRIZ DE CONFUSIÓN (Set de prueba, Random Forest)
=====
```

```
Verdaderos Positivos (TP): 23
Verdaderos Negativos (TN): 202
Falsos Positivos (FP): 24
Falsos Negativos (FN): 36
```

```
Accuracy: 78.95%
F1-Score: 43.40%
Recall: 38.98%
Precision: 48.94%
```

A misty forest scene with tall trees and sunlight filtering through the canopy. The text "¡GRACIAS!" is overlaid in the center.

¡GRACIAS!